1. ¿Cómo funciona Java? (JVM, JRE y JDK)

Proceso de compilación e interpretación:

Java combina compilación e interpretación. Primero escribes el código fuente en un fichero **.java**. El compilador (**javac**) lo traduce a **bytecode** (fichero **.class**), que es independiente del sistema operativo. Finalmente, la **JVM (Java Virtual Machine)** de cada máquina interpreta y ejecuta ese bytecode.

Plataforma y Sintaxis Java:

Lenguaje orientado a objetos (POO), claro, legible y multiplataforma (*Write Once, Run Anywhere*).

Diferencia entre JRE y JDK:

• **JRE (Java Runtime Environment):** Contiene la máquina virtual (JVM) y las librerías necesarias únicamente para **ejecutar** aplicaciones Java.
• **JDK (Java Development Kit):** Más completo; incluye todo el JRE más las herramientas para **desarrollar y compilar** (compilador *javac*, depurador, etc.).

Ejemplo práctico — HolaJVM.java

```
// 1. Compilar en terminal: javac HolaJVM.java  --> Genera HolaJVM.class (Bytecode)
// 2. Ejecutar en terminal: java HolaJVM        --> La JVM ejecuta el bytecode
public class HolaJVM {
    public static void main(String[] args) {
        System.out.println("¡Hola desde la Máquina Virtual de Java (JVM)!");
        System.out.println("Sistema Operativo: " + System.getProperty("os.name"));
        System.out.println("Versión de Java (JDK/JRE): " + System.getProperty("java.version"));
    }
}
```

IDEA CLAVE: *El archivo .java lo lee el programador; el archivo .class (bytecode) lo lee y ejecuta la JVM.*

2. Configuración del Entorno y Primer Programa (HelloWorld)

Requisitos para programar en Java:

1. Descargar e instalar la última versión estable del **JDK**.
2. Instalar **IntelliJ IDEA Community** (el IDE más utilizado en desarrollo Java).

Estructura básica de un proyecto Java:

• **Carpeta src:** Contiene los paquetes (*packages*) y las clases (*.java*).
• **Clase principal:** El nombre de la clase pública (*public class Main*) debe coincidir exactamente con el nombre del archivo (*Main.java*).
• **Método main:** Es el punto de entrada donde comienza la ejecución del programa: *public static void main(String[] args)*.

Salida por pantalla:

Se utiliza *System.out.println("texto")* para imprimir una línea en la consola.

Ejemplo práctico — Main.java (HelloWorld)

```
package org.example;

public class Main {
    // Punto de entrada de cualquier aplicación Java
    public static void main(String[] args) {
        System.out.println("¡Hola Mundo desde IntelliJ IDEA!");
        System.out.println("Mi primer proyecto en Acceso a Datos - 2º DAM");
    }
}
```

IDEA CLAVE: *Atajo útil en IntelliJ IDEA: escribe 'psvm' o 'main' + Tab para generar el método main, y 'sout' + Tab para System.out.println().*

3. Fundamentos de Java: Tipos, Colecciones, POO, Lambdas y Excepciones

Tipos primitivos, Wrappers, Constantes y Enum:

Tipos básicos (*byte, short, int, long, float, double, char, boolean*) y sus clases equivalentes (*Integer, Double, Boolean*) para conversiones (*Integer.parseInt()*). Constantes con *static final* y conjuntos fijos con *enum*.

Colecciones principales (java.util):

• **List (ArrayList, LinkedList):** Secuencia ordenada que permite elementos duplicados.
• **Map (HashMap<K, V>):** Pares clave-valor donde la clave es única (*put(), get()*).
• **Set (HashSet):** Conjunto desordenado que NO permite duplicados.

Expresiones Regulares, POO, Interfaces y Excepciones:

• **Regex:** Validación de patrones con *cadena.matches("expresion")*.
• **POO e Interfaces:** Herencia (*extends*), clases abstractas, interfaces (*implements*) y expresiones **lambda** *(a, b) -> a + b*.
• **Excepciones:** *Checked* (obligatorio capturarlas, ej. *IOException*) vs *Unchecked* (en ejecución, ej. *NullPointerException*) con bloques *try-catch-finally*.

Ejemplo práctico — EjemploColeccionesPOO.java

```
package Ejemplos;
import java.util.*;

public class EjemploColeccionesPOO {
    @FunctionalInterface
    interface Operacion { int calcular(int a, int b); }

    public static void main(String[] args) {
        // 1. ArrayList (admite duplicados) y HashSet (sin duplicados)
        ArrayList<String> lista = new ArrayList<>(List.of("Java", "XML", "Java"));
        HashSet<String> conjunto = new HashSet<>(lista);

        // 2. HashMap (Clave -> Valor)
        HashMap<Integer, String> mapa = new HashMap<>();
        mapa.put(1, "El principito");
        System.out.println("Mapa ID 1: " + mapa.get(1));

        // 3. Regex y Lambda
        boolean emailOk = "alumno@ies.com".matches("^[\\w.-]+@[\\w.-]+\\.[a-zA-Z]{2,}$");
        Operacion suma = (a, b) -> a + b;
        System.out.println("Email OK: " + emailOk + " | Suma: " + suma.calcular(5, 10));

        // 4. Excepción controlada con try-catch-finally
        try {
            int n = Integer.parseInt("abc");
        } catch (NumberFormatException e) {
            System.out.println("Excepción capturada: formato incorrecto.");
        }
    }
}
```

IDEA CLAVE: *Usa ArrayList para listas ordenadas, HashSet para evitar repetidos y HashMap para búsquedas rápidas por clave.*

4. Flujos de Entrada/Salida (Streams): Binarios y de Caracteres

Concepto de Stream (Flujo) y E/S Estándar:

Un flujo de entrada (*Input*) lee datos desde un origen y uno de salida (*Output*) los escribe en un destino. Java incluye *System.in* (teclado), *System.out* (consola) y *System.err* (errores).

1. Flujos Binarios (orientados a bytes):

• **FileOutputStream / FileInputStream:** Escriben y leen bytes crudos (*read()* devuelve -1 al llegar al final).
• **BufferedOutputStream / BufferedInputStream:** Usan un búfer intermedio para reducir accesos a disco.
• **DataOutputStream / DataInputStream:** Guardan y leen tipos primitivos (*writeInt, writeDouble, readInt, readDouble*). **Es obligatorio leerlos en el mismo orden exacto en que se escribieron.**

2. Flujos de Caracteres (orientados a texto):

• **FileWriter / FileReader:** Escriben y leen carácter a carácter.
• **BufferedWriter / BufferedReader:** Permiten trabajar línea a línea (*readLine()* devuelve *null* al final del fichero).

Ejemplo práctico — EjemploFlujosCaracteres.java y EjemploFlujosBinarios.java

```
package Ejemplos;
import java.io.*;

public class EjemploFlujos {
    public static void main(String[] args) {
        // A. Escritura y lectura de TEXTO línea a línea con BufferedWriter/Reader
        try (BufferedWriter bw = new BufferedWriter(new FileWriter("texto.txt"))) {
            bw.write("Hola desde BufferedWriter");
            bw.newLine();
        } catch (IOException e) { e.printStackTrace(); }

        try (BufferedReader br = new BufferedReader(new FileReader("texto.txt"))) {
            String linea;
            while ((linea = br.readLine()) != null) {
                System.out.println("Texto leído: " + linea);
            }
        } catch (IOException e) { e.printStackTrace(); }

        // B. Escritura y lectura BINARIA de tipos primitivos con DataStreams
        try (DataOutputStream dos = new DataOutputStream(new FileOutputStream("datos.dat"))) {
            dos.writeInt(42);
            dos.writeDouble(99.95);
        } catch (IOException e) { e.printStackTrace(); }

        try (DataInputStream dis = new DataInputStream(new FileInputStream("datos.dat"))) {
            System.out.println("Int: " + dis.readInt() + " | Double: " + dis.readDouble());
        } catch (IOException e) { e.printStackTrace(); }
    }
}
```

IDEA CLAVE: *Regla de oro: Para archivos de texto (.txt, .csv) usa Reader/Writer; para archivos binarios (.dat, imágenes) usa InputStream/OutputStream.*

5. Clase File, Sentencia try-with-resources y Clase Path (NIO.2)

Clase java.io.File:

Representa la ruta de un archivo o carpeta (NO lee ni escribe su contenido). Métodos principales: *exists()*, *createNewFile()*, *mkdir()*, *renameTo()*, *delete()*, *isFile()*, *isDirectory()*, *length()* y *list() / listFiles()*.
Desventaja: mal manejo de errores (devuelve booleanos sin detallar la causa del fallo).

Sentencia try-with-resources:

Permite declarar los flujos entre paréntesis tras el *try (...)* para que Java los cierre automáticamente al terminar el bloque, sin necesidad de *finally*.

Clase Path y Files (java.nio.file):

Alternativa moderna a *File*. Se combina *Paths.get("ruta")* con los métodos estáticos de *Files*, como *Files.write(path, lista)*, *Files.readAllLines(path)*, *Files.newBufferedWriter()* y *Files.newBufferedReader()*.

Ejemplo práctico — EjemploFile.java y EjemploPathNIO.java

```
package Ejemplos;
import java.io.File;
import java.io.IOException;
import java.nio.file.*;
import java.util.List;

public class EjemploFileYPath {
    public static void main(String[] args) {
        try {
            // 1. Crear directorio y archivo con java.io.File
            File dir = new File("./mi_directorio");
            if (!dir.exists()) dir.mkdir();
            File arch = new File(dir, "datos.txt");
            if (!arch.exists()) arch.createNewFile();
            System.out.println("¿Es archivo?: " + arch.isFile() + " | Bytes: " + arch.length());

            // 2. Escribir y leer todas las líneas directamente con Path y Files (NIO)
            Path p = Paths.get("./mi_directorio/datos.txt");
            Files.write(p, List.of("Primera línea con Path", "Segunda línea con Files"));

            List<String> leidas = Files.readAllLines(p);
            for (String s : leidas) {
                System.out.println("Leído con NIO: " + s);
            }
        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

IDEA CLAVE: *Ventaja de Path y Files: Con una sola línea (Files.readAllLines o Files.write) resuelves lo que antes requería un bucle con BufferedReader.*

6. Acceso Aleatorio a Ficheros (RandomAccessFile)

Acceso Secuencial vs Acceso Aleatorio:

A diferencia de los flujos secuenciales, **RandomAccessFile** funciona como un gran array de bytes con un cursor o **puntero interno** que podemos desplazar a cualquier posición para leer o sobrescribir un dato concreto sin tocar el resto del fichero.

Modos y métodos del puntero:

• Modos de apertura: **"r"** (solo lectura) y **"rw"** (lectura y escritura).
• **getFilePointer():** devuelve la posición actual del puntero en bytes.
• **seek(long pos):** coloca el puntero exactamente en el byte indicado (empezando desde el byte 0).
• **length():** devuelve el tamaño total del fichero en bytes.

Cálculo de posición en registros de tamaño fijo:

Sabiendo que *int* ocupa **4 bytes** y *double* ocupa **8 bytes**, si un registro tiene *id (4B) + precio (8B) + stock (4B)*, cada registro mide **16 bytes**. Fórmula del inicio del registro N: **(N - 1) * tamañoRegistro**.

Ejemplo práctico — EjemploRAF.java (Modificar un dato directo con seek)

```
package Ejemplos;
import java.io.IOException;
import java.io.RandomAccessFile;

public class EjemploRAF {
    private static final int TAMANO_REGISTRO = 16; // int(4) + double(8) + int(4) = 16 bytes

    public static void main(String[] args) {
        try (RandomAccessFile raf = new RandomAccessFile("videojuegosRAF.dat", "rw")) {
            // Guardamos 2 registros de 16 bytes cada uno
            raf.writeInt(1); raf.writeDouble(19.99); raf.writeInt(10); // Reg 1: bytes 0..15
            raf.writeInt(2); raf.writeDouble(49.99); raf.writeInt(5);  // Reg 2: bytes 16..31

            // Queremos modificar SOLO el stock del 2º videojuego (está en inicioRegistro + 12)
            long inicioRegistro2 = (2 - 1) * TAMANO_REGISTRO; // byte 16
            long posStock2 = inicioRegistro2 + 12;            // byte 28

            raf.seek(posStock2);
            System.out.println("Stock actual del juego 2: " + raf.readInt());

            raf.seek(posStock2); // Volvemos al byte 28 y sobrescribimos solo ese entero
            raf.writeInt(99);

            raf.seek(posStock2);
            System.out.println("Nuevo stock del juego 2: " + raf.readInt());
        } catch (IOException e) {
            System.out.println("Error RAF: " + e.getMessage());
        }
    }
}
```

IDEA CLAVE: *Cuidado: Cada vez que ejecutas readInt() o writeInt(), el puntero avanza 4 bytes automáticamente. Por eso hay que volver a hacer seek() antes de sobrescribir.*

7. Serialización de Objetos (Serializable y ObjectStreams)

¿Qué es serializar y deserializar?:

• **Serializar:** Convertir un objeto Java (o una colección *ArrayList* completa) en una secuencia de bytes para guardarlo en un fichero binario (*.dat*).
• **Deserializar:** Leer esos bytes del fichero y reconstruir los objetos originales en la memoria del programa.

La interfaz Serializable y serialVersionUID:

La clase cuyos objetos queremos guardar debe implementar **java.io.Serializable** e incluir el atributo *private static final long serialVersionUID = 1L;* (que comprueba que la versión de la clase coincide al leer el archivo).

Clases ObjectOutputStream y ObjectInputStream:

• **writeObject(objeto):** Guarda un objeto o un *ArrayList* entero en una sola llamada.
• **readObject():** Recupera el objeto desde el fichero (requiere hacer *casting* al tipo original y capturar *ClassNotFoundException*).

Ejemplo práctico — EjemploSerializacion.java

```
package Ejemplos;
import java.io.*;
import java.util.ArrayList;

class Videojuego implements Serializable {
    private static final long serialVersionUID = 1L;
    private int id; private String titulo; private double precio;

    public Videojuego(int id, String titulo, double precio) {
        this.id = id; this.titulo = titulo; this.precio = precio;
    }
    @Override
    public String toString() { return id + " - " + titulo + " - " + precio + " €"; }
}

public class EjemploSerializacion {
    public static void main(String[] args) {
        ArrayList<Videojuego> juegos = new ArrayList<>();
        juegos.add(new Videojuego(1, "Tetris", 19.99));
        juegos.add(new Videojuego(2, "Super Mario Bros", 29.99));

        // 1. Serializar la lista completa con writeObject()
        try (ObjectOutputStream out = new ObjectOutputStream(
                new FileOutputStream("juegos.dat"))) {
            out.writeObject(juegos);
        } catch (IOException e) { System.out.println(e.getMessage()); }

        // 2. Deserializar la lista haciendo casting
        try (ObjectInputStream in = new ObjectInputStream(
                new FileInputStream("juegos.dat"))) {
            ArrayList<Videojuego> recuperados = (ArrayList<Videojuego>) in.readObject();
            for (Videojuego v : recuperados) System.out.println(v);
        } catch (IOException | ClassNotFoundException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

IDEA CLAVE: *Si guardas un ArrayList, tanto ArrayList como Videojuego deben implementar la interfaz Serializable.*

8. Ficheros XML: Estructura y Lectura con DOM y SAX

Estructura y reglas de un XML bien formado:

Compuesto por declaración XML opcional, un **único elemento raíz**, elementos hijos, **atributos** entre comillas y contenido texto.
Para estar **bien formado**: toda etiqueta abierta debe cerrarse, no pueden cruzarse, distingue mayúsculas/minúsculas y solo hay una raíz.

Lectura con DOM (Document Object Model):

Carga todo el documento XML en memoria como un **árbol de nodos** (*Document*). Permite localizar etiquetas con *getElementsByTagName()*, leer atributos con *getAttribute()* y extraer su texto con *getTextContent()*.

Lectura con SAX (Simple API for XML):

Recorre el XML secuencialmente generando **eventos** sin cargarlo entero en memoria. Se extiende *DefaultHandler* sobrescribiendo *startElement()*, *characters()* y *endElement()*.

Ejemplo práctico — EjemploXML_DOM_SAX.java (Lectura con DOM y con SAX)

```
package Ejemplos;
import java.io.File;
import javax.xml.parsers.*;
import org.w3c.dom.*;
import org.xml.sax.Attributes;
import org.xml.sax.helpers.DefaultHandler;

public class EjemploXML_DOM_SAX {
    public static void main(String[] args) {
        // A. LECTURA CON DOM (Árbol en memoria)
        try {
            Document doc = DocumentBuilderFactory.newInstance()
                    .newDocumentBuilder().parse(new File("libros.xml"));
            NodeList libros = doc.getElementsByTagName("libro");
            for (int i = 0; i < libros.getLength(); i++) {
                Element libro = (Element) libros.item(i);
                String titulo = libro.getElementsByTagName("titulo").item(0).getTextContent();
                System.out.println("DOM -> ID: " + libro.getAttribute("id") + " | " + titulo);
            }
        } catch (Exception e) { System.out.println("Error DOM: " + e.getMessage()); }

        // B. LECTURA CON SAX (Eventos secuenciales)
        try {
            SAXParser analizador = SAXParserFactory.newInstance().newSAXParser();
            analizador.parse(new File("libros.xml"), new DefaultHandler() {
                private StringBuilder texto = new StringBuilder();
                public void startElement(String u, String l, String q, Attributes a) {
                    texto.setLength(0);
                }
                public void characters(char[] ch, int s, int len) { texto.append(ch, s, len); }
                public void endElement(String u, String l, String q) {
                    if (q.equals("titulo")) {
                        System.out.println("SAX -> Título: " + texto.toString().trim());
                    }
                }
            });
        } catch (Exception e) { System.out.println("Error SAX: " + e.getMessage()); }
    }
}
```

IDEA CLAVE: *DOM carga todo el XML como un árbol en memoria (permite buscar y escribir); SAX lo lee de principio a fin por eventos.*